

Persiapan Presentasi Docker

Mentoring: DevOps. Dokumen ini memetakan 7 poin wajib presentasi ke bukti konkret di project IOMS (file dan nomor baris), menjelaskan Dockerfile, dan berisi urutan demo yang bisa langsung dijalankan.

PROJECT IOMS (ioms/)	DISUSUN OLEH Arief Aryudi Syidik	TANGGAL 2 Oktober 2026
-------------------------	-------------------------------------	---------------------------

Ringkasan 7 Poin Wajib

#	POIN	BUKTI / CARA MENUNJUKKAN	STATUS
1	Docker terpasang	<code>docker --version</code> , <code>docker compose version</code> , <code>docker run hello-world</code>	Siap
2	Tunjukkan Dockerfile	Dockerfile (50 baris, base <code>php:8.2-apache</code>)	Siap
3	Jelaskan instruksi utama	Bagian "Penjelasan Dockerfile" di bawah	Siap
4	Build, run, status Up, browser, log	Bagian "Demo A"	Siap
5	Jalankan dengan Docker Compose	<code>docker-compose.yml</code> , <code>docker compose up --build</code>	Siap
6	Compose build dari Dockerfile	<code>docker-compose.yml:3-5</code> (<code>build: context: .</code>)	Siap
7	Database sebagai service + terhubung	<code>docker-compose.yml:22-40</code> (service <code>mysql</code>), <code>DB_HOST: mysql</code>	Siap

Catatan poin 6: brief menyebut `build: ..` karena contoh `compose.yml` berada di subfolder. Di IOMS, file `compose` berada di root repo, sehingga konteks yang setara adalah `build: context: .` dengan `dockerfile: Dockerfile`. Jelaskan ini jika ditanya.

1. Docker Terpasang

```
docker --version           # Docker version 27.4.0
docker compose version
docker run --rm hello-world
docker info | head -5
```

2. Dockerfile

Lokasi: `Dockerfile` di root repo. Buka dengan `cat -n Dockerfile` atau di editor agar nomor baris terlihat.

3. Penjelasan Instruksi Dockerfile

BARIS	INSTRUKSI	FUNGSI
-------	-----------	--------

1	<code>FROM php:8.2-apache</code>	Base image resmi: PHP 8.2 sudah terpasang bersama web server Apache.
5-12	<code>RUN apt-get ... docker-php-ext-install pdo pdo_mysql mysqli; a2enmod rewrite</code>	Memasang paket sistem dan ekstensi PHP untuk koneksi MySQL, mengaktifkan mod_rewrite untuk front-controller, lalu membersihkan cache apt agar image kecil. Digabung dalam satu <code>RUN</code> supaya hanya satu layer.
15	<code>ENV APACHE_DOCUMENT_ROOT</code>	Menetapkan variabel lingkungan untuk root web ke <code>public/</code> .
16-17	<code>RUN sed ...</code>	Mengganti DocumentRoot bawaan Apache ke <code>public/</code> sehingga hanya folder publik yang terekspos.
20-26	<code>RUN { echo ... } > ioms.conf; a2enconf</code>	Mengizinkan <code>.htaccess</code> (AllowOverride All) agar aturan rewrite router berfungsi.
29	<code>COPY --from=composer:2</code>	Multi-stage copy: mengambil binary Composer dari image resmi tanpa memasang manual.
31	<code>WORKDIR /var/www/html</code>	Direktori kerja untuk instruksi berikutnya dan saat container jalan.
34-36	<code>COPY composer.json composer.lock + RUN composer install</code>	Memasang dependensi lebih dulu supaya layer ini ter-cache selama composer.json tidak berubah (build ulang lebih cepat).
41	<code>COPY . .</code>	Menyalin seluruh source code aplikasi ke image.
44	<code>RUN composer dump-autoload</code>	Membuat autoloader yang optimal terhadap source final.
46-48	<code>RUN mkdir ... chown ... chmod</code>	Menyiapkan folder upload dan memberi kepemilikan ke <code>www-data</code> agar Apache dapat menulis.
50	<code>EXPOSE 80</code>	Dokumentasi bahwa container mendengarkan port 80 (publish tetap lewat <code>-p</code> atau Compose).

4. Demo A: Jalankan dari Dockerfile (docker build / run)

Container tunggal butuh MySQL. Untuk demo A, jalankan MySQL sementara di network yang sama, lalu aplikasi dari image hasil build.

```
# 1. Build image dari Dockerfile
docker build -t ioms-app:demo .

# 2. Network + database pendukung
docker network create ioms-net
docker run -d --name ioms-db --network ioms-net \
  -e MYSQL_ROOT_PASSWORD=root_change_me -e MYSQL_DATABASE=ioms \
  -e MYSQL_USER=ioms_user -e MYSQL_PASSWORD=change_me \
  -v "$PWD/database/schema.sql:/docker-entrypoint-initdb.d/01-schema.sql:ro" \
  -v "$PWD/database/seed.sql:/docker-entrypoint-initdb.d/02-seed.sql:ro" mysql:8.0

# 3. Run aplikasi
docker run -d --name ioms-demo --network ioms-net -p 8081:80 \
  --env-file .env -e DB_HOST=ioms-db -e DB_PORT=3306 ioms-app:demo

# 4. Container berstatus Up
docker ps --filter name=ioms

# 5. Akses di browser
open http://localhost:8081

# 6. Log tanpa fatal error
docker logs ioms-demo
docker logs ioms-demo 2>&1 | grep -i fatal || echo "tidak ada fatal error"

# Bersihkan sebelum demo B
docker rm -f ioms-demo ioms-db; docker network rm ioms-net
```

Tunggu sekitar 20-30 detik setelah MySQL dijalankan agar inisialisasi schema selesai sebelum membuka browser.

5-7. Demo B: Docker Compose

```
# Menjalankan app + mysql sekaligus (build dari Dockerfile)
docker compose up --build -d
docker compose ps          # app = Up, mysql = Up (healthy)
open http://localhost:8080
docker compose logs app | grep -i fatal || echo "tidak ada fatal error"
```

POIN	BUKTI DI DOCKER-COMPOSE.YML
6. Build dari Dockerfile	Baris 3-5: <code>build: context: ., dockerfile: Dockerfile</code> ; baris 6: <code>image: ioms-app:local</code>
7. Database sebagai service	Baris 22-40: service <code>mysql</code> (<code>mysql:8.0</code>), volume <code>db_data</code> , init <code>schema.sql</code> + <code>seed.sql</code> , healthcheck <code>mysqladmin ping</code>
Aplikasi mengenal DB	Baris 12-14: <code>DB_HOST: mysql</code> (nama service sebagai hostname), <code>DB_PORT: "3306"</code> ; dibaca oleh <code>app/Core/Database.php:23-26</code>
Urutan start	Baris 18-20: <code>depends_on</code> dengan <code>condition: service_healthy</code> , aplikasi menunggu MySQL siap

Menunjukkan aplikasi terhubung ke database:

```
# Dari sisi database: tabel dan data seed terisi
docker compose exec mysql mysql -uioms_user -pchange_me ioms -e "SHOW TABLES; SELECT
COUNT(*) FROM users;"

# Dari sisi aplikasi: koneksi PDO dari dalam container app
docker compose exec app php -r 'require "vendor/autoload.php"; App\Core\Env::load(".env");
echo App\Core\Database::connection()->query("SELECT DATABASE()")->fetchColumn(), PHP_EOL;'

# Bukti akhir di browser: login (data user dibaca dari MySQL), lalu buka Products /
Dashboard
```

Sesuaikan user/password dengan isi `.env` jika berbeda dari nilai default. Port host: aplikasi 8080, MySQL 3307 (mapping `3307:3306`).

Urutan Demo yang Disarankan

MENIT	LANGKAH
1	Versi Docker, <code>hello-world</code> (poin 1)
2-4	Buka Dockerfile, jelaskan instruksi (poin 2-3)
5-8	Demo A: build, run, <code>docker ps</code> , browser, logs (poin 4)
9-12	Demo B: <code>docker compose up --build</code> , tunjukkan <code>compose.yaml</code> dan service mysql, koneksi DB (poin 5-7)

Catatan Kejujuran / Antisipasi Pertanyaan

TEMUAN	PENJELASAN
Belum ada <code>.dockerignore</code>	<code>COPY . .</code> ikut membawa <code>.git</code> , <code>.env</code> , dan <code>vendor/</code> dari host ke image. Perbaikan: tambah <code>.dockerignore</code> .
Single-stage, root user	Image masih berisi git/unzip dan tidak ada instruksi <code>USER</code> ; Apache sendiri menurunkan hak ke <code>www-data</code> . Bisa diperbaiki dengan multi-stage build.
Bind mount <code>./:/var/www/html</code>	Di Compose, source host menimpa isi image (nyaman untuk development). Untuk produksi, hapus mount ini agar container memakai kode dari image.
Fallback <code>echo</code> pada composer install	Build tidak gagal bila offline, namun dapat menyembunyikan error dependensi; mengandalkan <code>vendor/</code> dari host.